Predictive Memory Paging System with Workload Awareness

G. K. Shreyas Bharadwaj
Reg. No. 24BCE2256
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Vaastav Bajanthri
Reg. No. 24BCE2223
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

P. Giphy Bershan
Reg. No. 24BCE2201
Dept. of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*—Operating systems keep the pages a program is using in a small, fast main memory and the rest on a slow disk; a request for a non-resident page is a page fault. Classical paging reacts to faults and only chooses which page to evict. This paper studies predictive paging: guessing the next pages and loading them early, together with a decision engine that decides when a guess is worth the risk, because on a full memory every prefetch evicts a page and a wrong guess can cause a fault. We built a trace-driven simulator with six eviction policies, six predictors and nine decision engines, and evaluated them on 22 real benchmark traces (SPEC CPU 2006 and GAP) at two memory sizes (44 configurations). Confidence and prefetch precision proved to be misleading feedback signals; the measured fault delta (faults without prefetching minus faults with prefetching) works best, and we obtain it exactly and cheaply with a shadow directory that stores page identifiers only and can be hash-sampled. The final system, Linux-style readahead combined with a delta-Markov predictor and steered by shadow-directory feedback, removes on average 55.0% of LRU's page faults (readahead alone 50.7%, Markov prediction alone 7.7%) and is worse than LRU in only 1 of 44 configurations. An I/O time model estimates a 1.43× speed-up over LRU on a parallel SSD, level with readahead (1.44×), while on a single-channel hard disk every prefetcher is slower than LRU: fewer faults do not automatically mean faster programs.**

***Index Terms*—virtual memory, demand paging, page replacement, prefetching, Markov predictor, readahead, feedback control, trace-driven simulation.**

I. Introduction

Main memory (RAM) is fast but small, while secondary storage is large but thousands of times slower. The operating system divides memory into fixed-size pages (4 KB in this work) and keeps only the pages in use in RAM, in slots called frames. When a program requests a non-resident page, a page fault occurs and the program waits while the page is read from disk. When RAM is full, the page replacement (eviction) policy chooses a resident page to remove [1], [2].

Classical policies such as FIFO, LRU and Clock [3] are reactive: they act only after a fault. Belady's optimal policy (OPT) [4], which evicts the page needed furthest in the future, is the best any eviction policy can do, but it needs knowledge of the future and still pays for the first access to every page (a compulsory fault).

If the operating system could predict the next pages, it could load them before they are requested (prefetching) and avoid faults, including compulsory faults that no eviction policy can avoid. Prefetching is, however, a bet: on a full memory every prefetched page evicts another page, so a wrong guess can evict a page that is needed soon and create a fault that would not otherwise have happened. A useful predictive paging system therefore needs both a predictor and a component that decides when acting on a prediction is worth the risk.

We design and evaluate a predictive paging system that (i) predicts future page requests cheaply from the access stream, (ii) decides for each prediction whether to prefetch, using feedback on whether prefetching is actually reducing page faults, and (iii) is compared fairly against classical and modern baselines on real program traces, including an estimate of whether fewer faults translate into shorter run time. Our contributions are:

- an evaluation of feedback signals for cost-aware prefetching at page granularity: confidence cannot discriminate, precision and ghost-list estimates can mislead, and the measured fault delta works;
- a shadow directory that provides the exact fault delta at about 0.2% of memory and can be hash-sampled without changing decisions in our tests;
- a combination of readahead and delta-Markov prediction with the lowest fault count of all systems tested;
- a time model and break-even analysis showing when fewer faults lead to faster programs, and documented negative results with their causes.

Section II reviews related work, Section III describes the design, Section IV the implementation and method, Section V verification, Section VI the results, and Section VII concludes.

II. Related Work

A. Page Replacement

Belady [4] defined the optimal replacement policy, used here as a lower bound. LRU, FIFO and the Clock (second-chance) approximation of LRU are the standard practical policies [2], [3]. LRU-K [5] evicts the page whose K-th most recent access is oldest, which resists one-time scans; we implement LRU-2. ARC [6] balances recency and frequency adaptively using "ghost" lists of recently evicted page identifiers.

B. Adapting to the Fault Rate

The Page Fault Frequency algorithm [7] adjusts the memory allocated to a process according to its measured fault rate. Our decision engine uses a related idea, feedback on faults, but applies it to the prefetching threshold and measures the fault difference caused by prefetching rather than the fault rate.

C. Prefetching

Joseph and Grunwald [8] introduced prefetching with Markov predictors, which learn the probability of the next

address given the current one; our Markov predictor is a bounded version of this idea. The Linux page cache fetches a growing window of pages when it detects sequential access [9]; we implement a Linux-style readahead as a baseline. PACMan [10] showed that prefetched lines should be inserted with low priority so that wrong prefetches are evicted early. Feedback-directed prefetching [11] adjusts how aggressively a hardware prefetcher runs using its measured accuracy, lateness and cache pollution. Linux's workingset code keeps "shadow entries" for evicted pages to detect refaults [12], and SHARDS [13] samples page identifiers by hash to estimate cache behaviour with a small fraction of the memory; we use it to shrink our shadow directory. At the operating-system level, Canvas [14] adapts swap prefetching to each application, KML [15] tunes the Linux readahead size online with a small in-kernel model, and CrossPrefetch [16] exposes prefetch effectiveness so that a runtime can adapt how much it prefetches.

D. Recent Work and Research Gap

TABLE I
RECENT WORK AND HOW THIS PAPER DIFFERS

Ref.	Focus	Gap relative to this work
[17]	database buffer prefetching	specific to database structures
[18]	learned page replacement	eviction only, no prefetching
[19]	cross-page spatial prefetching	spatial patterns only
[20]	GPU unified-memory prefetching	overhead when workload deviates; no cost control
[21]	attention-based graph prefetcher	graph-specific, no cost awareness
[22]	runtime prefetcher selection	hardware caches, not OS pages
[23]	indirect-access prefetching	no system-wide cost awareness
[24]	in-memory database indexing	needs application knowledge
[25]	application-aware virtual memory	requires modifying the application
[26]	profile-guided prefetch generation	struggles with phase changes

Most prior work proposes a better predictor or a better eviction policy and reports hit rates or fault counts. Less attention is given to (a) which feedback signal should control a prefetcher so that it does not cause harm under memory pressure, (b) how to obtain that signal cheaply in a real system, and (c) whether fewer faults actually mean shorter run time on a given device. This paper addresses these three questions. Its building blocks (Markov prediction, readahead, fault-based feedback, ghost and shadow lists) are known; the contribution lies in their combination and evaluation.

III. SYSTEM DESIGN

A. Overview

The system is a trace-driven simulator (Fig. 1). A recorded list of page requests is replayed one request at a time. For each request the memory engine reports a hit or a fault; the predictor learns from the request and proposes candidate pages; the decision engine accepts or rejects each candidate; accepted candidates are loaded into memory. Periodically a feedback controller compares the faults of the real system with those of a no-prefetch shadow and adjusts how cautious the decision engine is. Table II lists the modules.

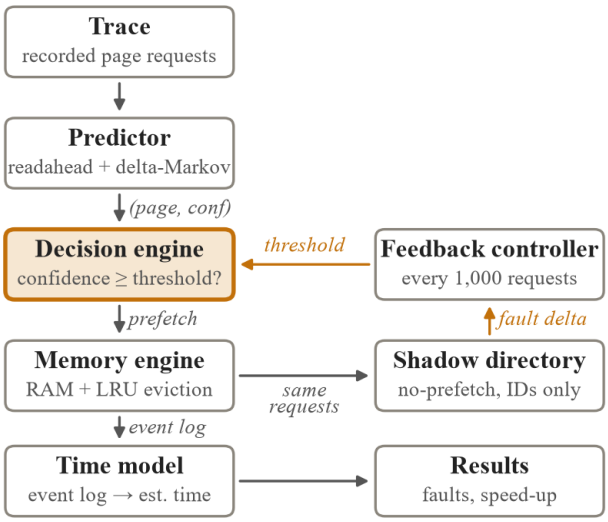


Fig. 1. System architecture. The predictor proposes (page, confidence) candidates; the decision engine admits them; the shadow directory supplies the fault delta that sets the threshold.

TABLE II
MODULES OF THE SYSTEM

Module (file)	Responsibility	Output
Traces (traces.py)	generate, convert and load traces	page sequence, 25% warm-up split
Memory engine (engine.py)	model RAM; single path by which pages enter memory	hit/fault, counters, event log
Eviction policies (engine.py)	choose which page leaves	victim page
Predictors (predictors.py)	guess the next pages	(page, confidence) candidates
Decision engines (deciders.py)	accept or reject candidates; adapt the threshold	yes/no
Simulators (simulate.py)	per-request loop with the chosen feedback	statistics
Time model (timing.py)	turn events into estimated time	estimated run time
Final system (final.py)	frozen configuration of the best system	statistics

B. Per-Request Processing

1) Receive the next requested page p.
2) Memory engine: if p is resident, record a hit; otherwise record a fault, evict a victim if memory is full, and load p.
3) Predictor: learn the transition just observed.
4) Every epoch (1,000 requests): compute the fault delta from the shadow directory and move the decision threshold.
5) Predictor: propose candidates for the next request.
6) Decision engine: decide for each non-resident candidate whether to prefetch it.
7) Load approved candidates through the same path as demand faults.

Two design rules matter. First, prefetched pages enter memory only through the same install routine as demand faults, so a faulty predictor can make the system slower but never incorrect. Second, the predictor sees only the request history and the decision engine sees only (page, confidence, memory state), so decision engines can be compared on identical predictions.

C. Predictors

TABLE III
PREDICTORS

Predictor	Mechanism	Weakness
Stride	if the last three gaps are equal, continue the gap	irregular access
Markov (order 1/2)	next-page counts after the last one/two pages; 256 contexts, 2 successors, counts halved every 25,000 updates	cannot predict unseen pages
Hybrid	order 2, then order 1, then stride	page-number based
Delta-Markov	Markov table over gaps between pages	random access
Readahead	two consecutive pages start a stream; window doubles at its midpoint, capped at RAM/4	jumping access
Final	readahead window first, then the delta-hybrid guess	more prefetch reads

D. Decision Engine

When free frames exist, any candidate with confidence of at least 0.15 is accepted, since a wrong prefetch then evicts nothing. When memory is full, a candidate is accepted only if its confidence is at least an adaptive threshold $\theta \in [0.15, 0.95]$. Every epoch of length E the controller computes the delta rate

$$\Delta = (F_{\text{noPF}} - F_{\text{PF}}) / E \quad (1)$$

where the two terms are the faults without and with prefetching. If $\Delta > +0.002$, θ is lowered by 0.1 (prefetch more); if $\Delta < -0.002$ it is raised by 0.1 (prefetch less); otherwise it is unchanged.

E. Obtaining the Fault Delta

Table IV compares three ways of measuring the faults the system would have had without prefetching. The shadow directory is adopted.

TABLE IV
WAYS OF MEASURING THE FAULT DELTA

Method	How "faults without prefetching" is obtained	Deployable?
Oracle	full second copy of memory runs the same requests without prefetching	no
Ghost list	remember pages a prefetch evicted (256 entries); count refaults within 2,000 requests	yes (~4 KB), biased on some workloads
Shadow directory	no-prefetch copy of the eviction policy holding page IDs only, optionally hash-sampled	yes (8 B per frame, ~0.2% of RAM)

F. Time Model

Each run can record an event log of demand faults, prefetches and first uses of prefetched pages. The time model replays it: every request costs a compute time (swept over 0.1, 1 and 10 µs because it is unknown for these traces) and a prediction cost (0.2 µs); every read takes 80 µs on an NVMe SSD with 8 parallel channels or 5 ms on a hard disk with one channel; demand faults block, possibly behind earlier prefetch reads; a prefetched page used before its read completes stalls the program for the remainder.

IV. IMPLEMENTATION AND EXPERIMENTAL METHOD

A. Implementation

All code is Python [27] (3.9 or later; developed on Python 3.14 under Windows 11, early stages on Google Colab [28]) using NumPy [29], pandas [30], matplotlib [31] and pytest [32]. The standard library supplies trace decompression (lzma), download (urllib) and the core data structures: ordered dictionaries for LRU, ARC and the Markov tables, heaps for LRU-2 and the disk channels, and binary search for OPT. The library (src/paging) is driven by one script per experiment; the code is available at github.com/shrey1-1/Predictive-memory-paging-system-.

- **Memory engine.** Resident pages are kept in a set and the policy object keeps its own ordering. A single install routine adds pages, evicts a victim if memory is full, and marks prefetched pages so that their later use or waste can be counted.
- **LRU and OPT.** LRU uses an ordered dictionary (a hit moves the page to the end; the victim is the first entry). OPT precomputes the positions of every page and binary-searches each resident page's next use.
- **Markov predictor.** At most 256 contexts (least recently used evicted), two successors per context, counts halved every 25,000 updates. Confidence is the best successor's share of the counts; with two successors it is never below 0.5.
- **Final system.** Readahead (confidence = run/(run+1)) plus the delta-hybrid Markov predictor, the shadow-directory decider with the frozen controller (epoch 1,000, step 0.1, dead band 0.002, no prefetch budget cap) and LRU.

B. Workloads and Memory Sizes

We use 22 real traces from the ML-Based Data Prefetching Competition LoadTraces [33], distributed for the ChampSim simulator [34]: SPEC CPU 2006 [35] (bwaves s0–s2, leslie3d s0–s2, milc, libquantum, mcf, soplex, omnetpp, astar, GemsFDTD, lbm, sphinx3) and GAP [36] (bfs-3, bfs-10, pr-3, pr-10, bc-3, cc-5, sssp-3), 200,000 requests each, with addresses mapped to pages by a 12-bit shift. Six synthetic traces with known structure serve as correctness tests. Memory is set to 1% and 5% of each trace's working set (at least 8 frames), giving 44 configurations; with large fixed memories every policy behaves identically.

C. Tuning and Metrics

Every parameter was chosen using only the first 25% of each trace, with the selection rule written down before each sweep, and then frozen. The main metric is the fault reduction relative to LRU,

$$\text{vs_LRU} = (F_{\text{LRU}} - F) / F_{\text{LRU}} \quad (2)$$

which is positive when a system has fewer faults than LRU and negative when prefetching created extra faults. We also report the useful prediction rate (coverage × accuracy), the break-even prefetch cost, and the estimated speed-up time(LRU)/time(system).

V. VERIFICATION

The correctness of a simulator cannot be judged by its results alone, so three kinds of evidence are used: automated tests that

encode properties any correct implementation must satisfy (Table V), independent re-implementations, and exact reproduction of earlier results. All 24 tests pass.

TABLE V
AUTOMATED TEST CASES

Group	Test cases	Type
Invariants	no policy beats OPT; LRU and OPT never worsen with more memory; with ample memory, faults = distinct pages; a sequential scan faults on every request	normal, boundary
Equivalence	ghost-list engine equals plain engine without prefetching; new options switched off reproduce old behaviour exactly	regression
Pinned numbers	fault counts of every policy, decider and the final system on a fixed synthetic trace	regression
Components	ARC ghost lists bounded; delta-Markov predicts unseen pages; full shadow directory equals the true delta; time model charges late prefetches	normal
Invalid use	a ghost-list decider passed to the plain simulator is rejected	invalid input

Independent implementations of LRU, FIFO and OPT gave identical fault counts on 40 random traces, and an independent LRU-2 matched on three real traces, confirming that LRU-2's poor results are genuine. The decision-engine ablation rerun on freshly downloaded traces reproduced all 48 rows of the original results exactly. Testing also exposed three problems, all fixed: a ghost-list decider passed to the wrong simulator silently behaved as the oracle; "confidence is never below 0.5" turned out to be a structural property of keeping two successors, not of the workloads; and a 35% prefetch budget cap blocked the system on omnetpp while it was winning every epoch, so it was removed after a warm-up-only check.

VI. RESULTS AND DISCUSSION

All results are page-fault counts or quantities derived from them. Traces are deterministic, so each configuration was run once; robustness comes from many traces and memory sizes rather than repeated runs.

A. Prediction Quality

TABLE VI
USEFUL PREDICTION RATE (COVERAGE × ACCURACY)

Trace	Stride	Markov o1	Markov o2	Hybrid
bwaves-s0	25.6%	78.8%	76.1%	80.4%
leslie3d-s0	0.2%	84.3%	81.6%	84.4%
bfs-3	0.0%	66.9%	67.3%	70.6%
pr-3	0.0%	45.8%	88.1%	89.2%
clustered*	0.0%	9.7%	8.1%	9.9%
cyclic*	80.0%	100.0%	100.0%	100.0%
sequential*	100.0%	0.0%	0.0%	100.0%
random*	0.0%	0.0%	0.0%	0.0%

*Synthetic trace.

The hybrid predictor guesses the next page exactly 81.2% of the time on the four real traces. Markov tables score zero on the sequential trace because its working set exceeds the 256-entry table; the stride fallback covers it.

B. Eviction Policies Alone

TABLE VII
PAGE FAULTS WITHOUT PREFETCHING (RAM = 1% OF WORKING SET)

Trace	Frames	FIFO	LRU	Clock	OPT
clustered*	9	5,729	5,803	5,676	2,637
bwaves-s0	30	3,380	3,140	3,214	3,112
leslie3d-s0	21	15,608	15,313	15,126	5,881
bfs-3	58	6,959	6,766	6,802	6,179
pr-3	40	5,867	5,812	5,820	5,346

*Synthetic trace.

On leslie3d, OPT has 62% fewer faults than LRU, so there is large room for improvement; on bwaves there is almost none. ARC and LRU-2 (Section VI-E) did not close the gap on the real traces.

C. Decision Engines

TABLE VIII
FAULT REDUCTION VS. LRU BY DECISION ENGINE (ORIGINAL MARKOV PREDICTOR)

Config. (frames)	Always	Fixed 0.6	Fixed 0.85	Precision	Fault delta
leslie3d-s0 (21)	+35.00%	+33.59%	+28.90%	+34.13%	+34.98%
bwaves-s0 (30)	+37.29%	+37.29%	+37.29%	+37.29%	+37.29%
clustered* (6)	−0.87%	−0.98%	−0.81%	−0.96%	−0.72%
clustered* (9)	+0.09%	−0.93%	−0.19%	−1.22%	−0.48%
bfs-3 (20)	−1.50%	−0.41%	−0.05%	−0.45%	−0.28%
bfs-3 (58)	−0.27%	−0.06%	+0.01%	−0.06%	−0.19%
pr-3 (15)	+0.51%	+0.63%	+0.59%	+0.63%	+0.51%
pr-3 (40)	+0.74%	+0.69%	+0.67%	+0.69%	+0.74%

*Synthetic trace.

TABLE IX
SUMMARY OVER THE EIGHT CONFIGURATIONS

Decision engine	Mean gain	Worst case	Total harm
fault delta	+8.98%	−0.72%	−1.67%
fixed 0.85	+8.30%	−0.81%	−1.05%
fixed 0.6	+8.73%	−0.98%	−2.38%
precision	+8.76%	−1.22%	−2.69%
always	+8.87%	−1.50%	−2.64%
fixed 0.3	+8.87%	−1.50%	−2.64%

Confidence could not separate good from bad predictions: no prediction had confidence below 0.5, so a 0.3 threshold equals always prefetching. Precision misled: on the clustered trace four engines had precision of 81–86% while faults increased, because a used prefetch can still evict a page needed sooner. The fault-delta engine halved the worst case (−0.72% against −1.50% for always prefetching) while keeping almost all of the benefit.

D. Head-to-Head With Classical Policies

TABLE X
PAGE FAULTS WITH MARKOV PREDICTION ADDED TO LRU

Trace	Frames	FIFO	LRU	Clock	LRU + pred.	OPT
leslie3d-s0	21	15,608	15,313	15,126	**9,956**	5,881
bwaves-s0	30	3,380	3,140	3,214	**1,969**	3,112
clustered*	9	5,729	5,803	5,676	5,831	2,637
bfs-3	20	7,614	7,394	7,390	7,415	6,349
pr-3	15	5,945	5,905	5,933	5,875	5,591

*Synthetic trace. Memory sizes as in the original head-to-head runs.

Prediction removed 35% of LRU's faults on leslie3d and 37% on bwaves; on bwaves it even beats OPT, because prefetching avoids compulsory faults that OPT must pay. The ghost-list estimate matched the oracle (mean gain +9.03% against +8.98%).

E. Stronger Baselines

TABLE XI
PAGE FAULTS WITH ARC, LRU-2 AND LINUX-STYLE READAHEAD

Trace	LRU	ARC	LRU-2	Read-ahead	Markov	OPT
clustered* (9)	5,803	6,207	6,512	7,645	5,798	2,637
leslie3d-s0 (21)	15,313	16,553	26,188	11,624	9,956	5,881
bwaves-s0 (30)	3,140	3,147	5,460	204	1,969	3,112
bfs-3 (20)	7,394	7,368	7,969	2,924	7,427	6,349
pr-3 (15)	5,905	5,825	5,822	2,905	5,872	5,591

*Synthetic trace. Frames in parentheses.

Linux-style readahead beat Markov prediction on three of the four real traces because it fetches new pages in a sequential run, which a page-number Markov model cannot predict. This motivated combining the two predictors.

F. Final System on 22 Real Traces

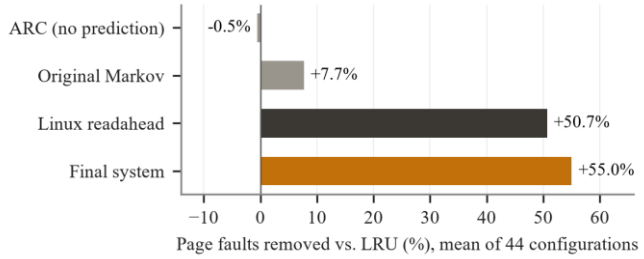


Fig. 2. Mean fault reduction versus LRU over 44 configurations (22 real traces × 2 memory sizes).

TABLE XII
FAULT REDUCTION VS. LRU, ALL 44 CONFIGURATIONS

System	Mean	Median	Worst	Worse than LRU
ARC (eviction only)	−0.5%	−0.1%	−8.2%	26 of 44
Original Markov	+7.7%	+1.5%	−0.3%	1 of 44
Linux-style readahead	+50.7%	+55.0%	−19.3%	2 of 44
Final system	**+55.0%**	**+59.3%**	**−3.3%**	**1 of 44**

The final system has fewer faults than readahead in 34 of 44 configurations, the same in 8 and more in 2, and fewer faults than OPT in 30. Its one loss to LRU is bwaves-s2 at 1% memory (8 frames). The largest loss to readahead is sphinx3, where the decision engine holds back useful readahead. Fig. 3 and Table XIII give the per-trace results at 1% memory.

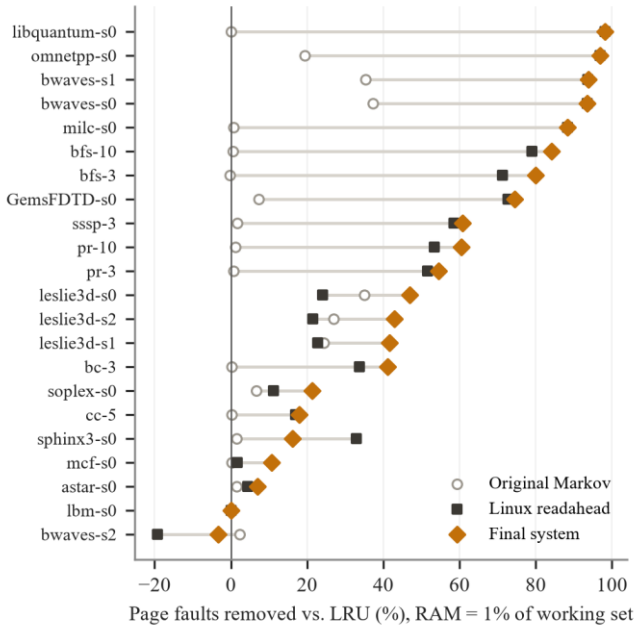


Fig. 3. Fault reduction per trace at 1% memory. Points to the right of zero are better than LRU.

TABLE XIII
PER-TRACE RESULTS AT 1% MEMORY

Trace	Fr.	LRU	OPT	Final	Final %	RA %	Mkv %
bwaves-s0	30	3,140	3,112	205	+93.5	+93.5	+37.3
bwaves-s1	30	3,140	3,111	195	+93.8	+93.6	+35.3
bwaves-s2	8	631	449	652	−3.3	−19.3	+2.4
leslie3d-s0	21	15,313	5,881	8,133	+46.9	+24.1	+35.0
leslie3d-s1	22	11,771	4,740	6,873	+41.6	+22.7	+24.4
leslie3d-s2	22	11,760	4,744	6,715	+42.9	+21.6	+27.0
milc-s0	232	23,269	23,269	2,700	+88.4	+88.4	+0.7
libquantum-s0	31	3,126	3,126	58	+98.1	+98.1	+0.0
mcf-s0	31	194,263	173,418	173,441	+10.7	+1.6	+0.2
bfs-3	58	6,766	6,179	1,356	+80.0	+71.3	−0.3
bfs-10	56	6,220	5,915	982	+84.2	+79.0	+0.6
pr-3	40	5,812	5,346	2,643	+54.5	+51.6	+0.7
pr-10	40	5,559	5,206	2,201	+60.4	+53.5	+1.2
soplex-s0	43	24,906	17,032	19,582	+21.4	+11.2	+6.7
omnetpp-s0	203	121,090	120,012	3,716	+96.9	+96.9	+19.4
astar-s0	48	110,547	82,922	102,867	+6.9	+4.3	+1.4
GemsFDTD-s0	29	3,760	3,314	960	+74.5	+72.7	+7.2
lbm-s0	31	3,136	3,128	3,136	+0.0	+0.0	+0.0
sphinx3-s0	20	13,425	10,501	11,263	+16.1	+33.0	+1.4
bc-3	23	31,644	30,762	18,635	+41.1	+33.8	+0.2
cc-5	13	141,398	129,935	116,086	+17.9	+16.9	+0.3
sssp-3	20	70,286	66,712	27,545	+60.8	+58.5	+1.6

Fr. = frames. The last three columns are the reduction vs. LRU (%) of the final system, Linux-style readahead (RA) and the original Markov system (Mkv).

G. Effect of the Feedback Signal

With the combined predictor, the ghost-list estimate reported "winning" on the synthetic clustered trace while the system was 21.5% worse than LRU. The shadow directory measures the true delta exactly; with it the same configuration is only 2.5% worse than LRU (2.4% with the frozen controller), and over the real traces it gives the best worst case of all deployable systems. Sampling one page in eight left fault counts unchanged on 9 of 10 traces tested at 5% memory.

H. Contribution of Each Component

TABLE XIV
PREDICTORS COMPARED, EVERYTHING ELSE FIXED (1% MEMORY)

Predictor	Exact next page	Mean vs. LRU	Worst	Worse than LRU	SSD speed-up
stride	2.7%	+5.8%	+0.0%	0	1.01×
markov-o1	50.1%	+4.9%	−0.8%	3	1.01×
markov-o2	53.3%	+2.4%	−1.1%	3	1.00×
hybrid	57.3%	+9.4%	−0.3%	1	1.01×
delta-hybrid	58.9%	+20.9%	+0.0%	0	1.02×
readahead	2.6%	+46.5%	−2.5%	1	1.51×
final	56.2%	**+51.2%**	−3.3%	1	1.49×

TABLE XV
DECISION ENGINES COMPARED, FINAL PREDICTOR (1% MEMORY)

Decision engine	Mean vs. LRU	Worst real trace	Clustered*	SSD speed-up
no prefetching	+0.0%	+0.0%	+0.0%	1.00×
always	+51.2%	−6.3%	−21.5%	1.49×
fixed 0.3	+51.2%	−6.3%	−21.5%	1.49×
fixed 0.6	+28.8%	+0.0%	−5.5%	1.21×
fixed 0.85	+19.8%	+0.0%	−0.2%	1.07×
precision	+26.8%	+0.0%	−5.2%	1.16×
fault-delta oracle	+49.1%	+0.0%	−1.7%	1.39×
ghost list	+47.9%	−3.2%	−21.5%	1.39×
shadow directory	+49.0%	−1.4%	−2.5%	1.39×
directory + frozen controller	**+51.2%**	−3.3%	**−2.4%**	1.49×

*Synthetic clustered trace.

Prediction accuracy and fault reduction rank the predictors in almost opposite order: readahead is exactly right about the next page only 2.6% of the time but removes the most faults of any single predictor, because it fetches pages never seen before. Always prefetching captures the full benefit on real traces but is badly hurt on the clustered trace; fixed thresholds are safe but lose half or more of the benefit; the final decision engine keeps the benefit and limits the harm.

I. Estimated Run Time

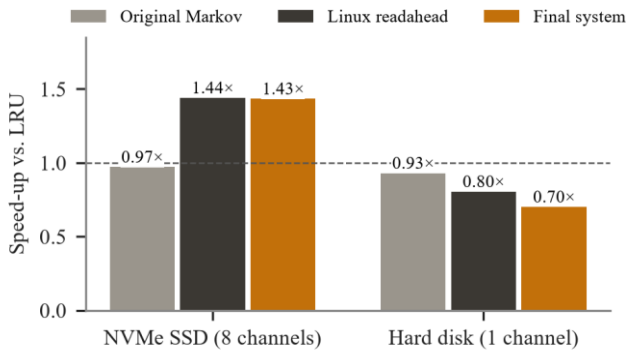


Fig. 4. Geometric-mean estimated speed-up versus LRU over 22 traces (1 µs compute per request, 0.2 µs prediction cost). Model estimates, not hardware measurements.

TABLE XVI
GEOMETRIC-MEAN ESTIMATED SPEED-UP VS. LRU

System	NVMe SSD (8 channels)	Hard disk (1 channel)
Original Markov	0.97× (slower on 18/22)	0.93×
Linux-style readahead	1.44× (slower on 2/22)	0.80×
Final system	1.43× (slower on 3/22)	0.70×

J. Discussion

- **Fewer faults are not the same as faster.** The final system removes the most faults, yet on the SSD model it only ties readahead, because the extra faults it removes cost extra prefetch reads.
- **Timeliness matters as much as accuracy.** Markov prediction alone removes 35–37% of faults on leslie3d and bwaves but is not faster: it prefetches one request ahead, so its pages often arrive late.
- **The storage device decides.** On a single-channel hard disk each prefetch costs as much as the fault it tries to save, so every prefetcher is slower than LRU.
- **The feedback signal matters.** Signals that count used prefetches as wins (precision, the ghost list) can report success while prefetching is causing harm; the measured fault delta does not.

VII. CONCLUSION AND FUTURE WORK

We built and evaluated a predictive paging system in a reproducible trace-driven simulator. On 22 real benchmark traces the final system, which combines Linux-style readahead with a delta-Markov predictor and steers prefetching with an exact fault-delta signal from a page-identifier-only shadow directory, removes on average 55.0% of LRU's page faults and is rarely harmful. The evaluation also shows the limits of fault counts as a measure: in estimated time the system is about 1.4 times faster than LRU on a parallel SSD, level with readahead, and slower than LRU on a hard disk.

Limitations. Cache-level traces mapped to pages approximate, but are not, operating-system paging, and the predictor sees every request, which a kernel does not. Time results come from a model with stated latencies and a swept compute time, not hardware measurements; demand-read priority, write-back and I/O merging are not modelled. Traces have 200,000 requests and memories are small (8 to 1,163 frames). ARC omits the textbook tie-break, LRU-2 has no correlated-reference period, and the readahead window is a quarter of memory.

Future work. Evaluate on operating-system page-fault traces or inside a kernel; add demand-read priority and prefetch-distance control and measure real hardware; compare with a learned predictor, such as an LSTM [37] or an attention-based model [21], under the same time model; validate shadow-directory sampling at realistic memory sizes; and recover the gains the cautious decision engine leaves on workloads such as sphinx3.

REFERENCES

[1] P. J. Denning, "Virtual memory," *ACM Comput. Surv.*, vol. 2, no. 3, pp. 153–189, 1970.

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[3] F. J. Corbató, "A paging experiment with the Multics system," in *In Honor of Philip M. Morse*, H. Feshbach and K. U. Ingard, Eds. Cambridge, MA, USA: MIT Press, 1969.

[4] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Syst. J.*, vol. 5, no. 2, pp. 78–101, 1966.

[5] E. J. O'Neil, P. E. O'Neil, and G. Weikum, "The LRU-K page replacement algorithm for database disk buffering," in *Proc. ACM SIGMOD*, 1993, pp. 297–306.

[6] N. Megiddo and D. S. Modha, “ARC: A self-tuning, low overhead replacement cache,” in *Proc. USENIX FAST*, 2003, pp. 115–130.

[7] W. W. Chu and H. Opderbeck, “The page fault frequency replacement algorithm,” in *Proc. AFIPS Fall Joint Comput. Conf.*, 1972, pp. 597–609.

[8] D. Joseph and D. Grunwald, “Prefetching using Markov predictors,” in *Proc. 24th Int. Symp. Comput. Archit. (ISCA)*, 1997, pp. 252–263.

[9] F. Wu, H. Xi, and C. Xu, “On the design of a new Linux readahead framework,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 75–84, 2008, doi: 10.1145/1400097.1400106.

[10] C.-J. Wu, A. Jaleel, M. Martonosi, S. C. Steely Jr., and J. Emer, “PACMan: Prefetch-aware cache management for high performance caching,” in *Proc. IEEE/ACM MICRO*, 2011, pp. 442–453.

[11] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, “Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers,” in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2007, pp. 63–74.

[12] J. Weiner, “Workingset detection (mm/workingset.c),” *Linux kernel source code*. [Online]. Available: https://github.com/torvalds/linux/blob/master/mm/workingset.c

[13] C. A. Waldspurger, N. Park, A. Garthwaite, and I. Ahmad, “Efficient MRC construction with SHARDS,” in *Proc. USENIX FAST*, 2015, pp. 95–110.

[14] C. Wang, Y. Qiao, H. Ma, S. Liu, Y. Zhang, W. Chen, R. Netravali, M. Kim, and G. H. Xu, “Canvas: Isolated and adaptive swapping for multi-applications on remote memory,” in *Proc. USENIX Symp. Netw. Syst. Des. Implement. (NSDI)*, 2023, pp. 161–179.

[15] I. U. Akgun, A. S. Aydin, A. Burford, M. McNeill, M. Arkhangelskiy, and E. Zadok, “Improving storage systems using machine learning,” *ACM Trans. Storage*, vol. 19, no. 1, Art. no. 9, 2023, doi: 10.1145/3568429.

[16] S. Garg, J. Zhang, R. Pitchumani, M. Parashar, B. Xie, and S. Kannan, “CrossPrefetch: Accelerating I/O prefetching for modern storage,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2024, pp. 102–116, doi: 10.1145/3617232.3624872.

[17] J. Yi, X. Wang, and P. Jin, “Workload-aware buffer prefetching for database systems,” *Data Sci. Eng.*, 2026, doi: 10.1007/s41019-025-00342-6.

[18] H. Kim, “Learning-based page replacement scheme for efficient I/O processing,” *Sci. Rep.*, vol. 15, Art. no. 4721, 2025.

[19] G. Vavouliotis, M. Torrents, B. Grot, K. Kalaitzidis, L. Peled, and M. Casas, “To cross, or not to cross pages for prefetching?,” in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2025, pp. 188–203, doi: 10.1109/HPCA61900.2025.00025.

[20] J. Rhee, I. Choi, G. Koo, Y. Oh, and M. K. Yoon, “Beyond VABlock: Improving transformer workloads through aggressive prefetching,” *J. Syst. Archit.*, 2025.

[21] P. Zhang, R. Kannan, A. V. Nori, and V. K. Prasanna, “Accelerating graph analytics using attention-based data prefetcher,” *SN Comput. Sci.*, vol. 5, no. 5, Art. no. 646, 2024.

[22] E. S. Alcorta, M. Madhav, R. Afoakwa, S. Tetrick, N. J. Yadwadkar, and A. Gerstlauer, “Characterizing machine learning-based runtime prefetcher selection,” *IEEE Comput. Archit. Lett.*, vol. 23, no. 2, pp. 146–149, 2024, doi: 10.1109/LCA.2024.3404887.

[23] F. Xue et al., “Tyche: An efficient and general prefetcher for indirect memory accesses,” *ACM Trans. Archit. Code Optim.*, vol. 21, no. 2, 2024, doi: 10.1145/3641853.

[24] Q. Zhang, H. Song, K. Zhou, J. Wei, and C. Xiao, “A prefetching indexing scheme for in-memory database systems,” *Future Gener. Comput. Syst.*, vol. 156, pp. 179–190, 2024, doi: 10.1016/j.future.2024.03.012.

[25] S. Jalalian, S. Patel, M. Rezaei Hajidehi, M. Seltzer, and A. Fedorova, “ExtMem: Enabling application-aware virtual memory management for data-intensive applications,” in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2024, pp. 397–408.

[26] Y. Zhang, N. Sobotka, S. Park, S. Jamilan, T. A. Khan, B. Kasikci, G. A. Pokam, H. Litz, and J. Devietti, “RPG2: Robust profile-guided runtime prefetch generation,” in *Proc. ACM Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS)*, 2024.

[27] Python Software Foundation, “Python programming language.” [Online]. Available: https://www.python.org

[28] Google, “Colaboratory.” [Online]. Available: https://colab.research.google.com

[29] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[30] W. McKinney, “Data structures for statistical computing in Python,” in *Proc. 9th Python Sci. Conf. (SciPy)*, 2010, pp. 56–61.

[31] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.

[32] H. Krekel et al., “pytest.” [Online]. Available: https://pytest.org

[33] ML-Based Data Prefetching Competition, “LoadTraces.” [Online]. Available: https://github.com/Quangmire/ChampSim

[34] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jiménez, E. Teran, S. Pugsley, and J. Kim, “The Championship Simulator: Architectural simulation for education and competition,” arXiv:2210.14324, 2022.

[35] J. L. Henning, “SPEC CPU2006 benchmark descriptions,” *ACM SIGARCH Comput. Archit. News*, vol. 34, no. 4, pp. 1–17, 2006.

[36] S. Beamer, K. Asanović, and D. Patterson, “The GAP benchmark suite,” arXiv:1508.03619, 2015.

[37] M. Hashemi, K. Swersky, J. A. Smith, G. Ayers, H. Litz, J. Chang, C. Kozyrakis, and P. Ranganathan, “Learning memory access patterns,” in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1919–1928.